Binary Bomb Analysis & Reverse Engineering Report

Analisi Tecnica Dettagliata delle Fasi di Decodifica x86/IDA Pro

TARGET BINARY	ARCHITETTURA	TOOL UTILIZZATI	STATO BOMBA
Bomb.exe (CMU Variant)	**x86 / 32-bit PE**	**IDA Pro, x32dbg, Python**	**COMPLETAMENTE DISARMATA**

Analisi Tecnica delle Singole Fasi

Fase 1: String Comparison (Confronto Diretto)

String Literal

Meccanismo di funzionamento: La prima fase verifica l'input dell'utente confrontandolo direttamente con una stringa costante memorizzata nella sezione di sola lettura del programma (`.rdata` / `.rodata`). La funzione `strings_not_equal` valuta i caratteri uno ad uno tramite un ciclo o la funzione standard `strcmp` .
Logica Assembly/Decompilatore: Un puntatore alla stringa dell'utente viene passato come primo parametro, mentre l'indirizzo della memoria staticamente definita viene passato come secondo. Se il valore restituito è diverso da 0, viene invocata `explode_bomb` .

```
Soluzione Fase 1: Public speaking is very easy.
```

Fase 2: Mathematical Sequence (Progressione Fattoriale)

Array & Loop

Meccanismo di funzionamento: La Fase 2 richiede l'inserimento di 6 numeri interi separati da spazio. Il programma utilizza `read_six_numbers` (basata su `sscanf` con formattatore `%d %d %d %d %d %d`). Il primo numero deve essere 1.
Logica Assembly/Decompilatore: Ogni elemento successivo `N[i]` viene calcolato iterativamente in base alla posizione: `N[i] = N[i-1] * (i + 1)` . Si tratta della sequenza dei fattoriali (1!, 2!, 3!, 4!, 5!, 6!).

```
Soluzione Fase 2: 1 2 6 24 120 720
```

Fase 3: Switch/Jump Table (Branching Condizionale)

Jump Table

Meccanismo di funzionamento: La funzione legge tre argomenti formattati tramite `sscanf` (un intero, un carattere, un intero). Il primo intero agisce da indice per un'istruzione di salto indiretto (Jump Table / `switch` case).
Logica Assembly/Decompilatore: In base all'indice scelto (es. `1`), l'esecuzione salta al caso corrispondente che impone la verifica specifica del secondo e terzo parametro. Per il caso 1, il secondo carattere deve corrispondere a `'b'` (ASCII 98) e il terzo numero a `214` .

```
Soluzione Fase 3: 1 b 214
```

Analisi Tecnica delle Singole Fasi (Continuazione)

Fase 4: Recursive Function (Serie di Fibonacci / Albero Ricorsivo)

Recursion

Meccanismo di funzionamento: Viene letto un singolo numero intero `v2` che deve soddisfare la condizione `v2 ≥ 1`. Il valore viene poi passato ad una funzione ricorsiva (`func4`).
Logica Assembly/Decompilatore: La funzione ricorsiva calcola la successione di Fibonacci `F(n) = F(n-1) + F(n-2)` con casi base `F(0)=0, F(1)=1`. La Fase 4 verifica che il risultato restituito dalla funzione sia esattamente uguale a `55` (oppure `521` a seconda della variante). Poiché `F(9) = 55`, l'input corretto è 55.

```
Soluzione Fase 4: 9
```

Fase 5: Array Transformation & Indexing (Scrambling Bitwise)

Bitwise & Lookup

Meccanismo di funzionamento: La funzione accetta una stringa di esattamente 6 caratteri. Ciascun carattere viene mascherato con un'operazione AND a 4 bit (`char & 0x0F`) per estrarre un valore numerico da 0 a 15.
Logica Assembly/Decompilatore: Il numero ottenuto viene usato come indice all'interno di una tabella di caratteri statica (`isrveawhobpnutfg`). La stringa risultante dopo la trasformazione deve corrispondere alla parola target `"giants"`. Mappando gli indici della parola target e convertendoli in caratteri ASCII validi si ottiene la stringa di input.

```
Soluzione Fase 5: opekma
```

Fase 6: Linked List Reordering (Puntatori e Ordinamento Nodi)

Data Structures

Meccanismo di funzionamento: La fase più complessa. Accetta 6 interi unici compresi tra 1 e 6. Legge una struttura dati di tipo Linked List composta da 6 nodi posizionati nella sezione `.data` della memoria.
Logica Assembly/Decompilatore: I numeri inseriti vengono usati per riordinare i puntatori `next` dei nodi della lista. Successivamente, un ciclo scorre la lista riordinata e controlla che ciascun valore contenuto nei nodi rispetti un ordinamento decrescente (`Nodo[i].valore ≥ Nodo[i+1].valore`). Estraendo i valori dei nodi dalla memoria e ordinandoli dal più grande al più piccolo si ricava l'ordine degli indici.

non riuscivo a farlo qundi ho fatto tutto di brute force

```
Soluzione Fase 6: 4 2 6 3 1 5
```

Tabella Riassuntiva delle Soluzioni

Fase	Tipo di Struttura / Algoritmo	Input Corretto	Note di Verifica
Fase 1	String comparison (`strcmp`)	`Public speaking is very easy.`	Stringa hardcoded in memoria
Fase 2	Loop iterativo (Fattoriale)	`1 2 6 24 120 720`	6 interi, `N[i] = N[i-1] * (i+1)`
Fase 3	Control Flow (Jump Table)	`1 b 214`	Case 1 della switch table
Fase 4	Algoritmo Ricorsivo (Fibonacci)	`9`	Calcola `F(7) = 21`
Fase			